Chapter 17 worked solutions

Designing Reproducible Evaluations

This guide covers the written solutions to Exercises 1–4 and 9–11. The companion Python notebook covers Exercises 5–8.

1. Choose an evaluation unit

The short conversation contributes one correct check out of one. The longer conversation contributes four correct checks out of five. To count every check equally, add the correct checks and divide by the total number of checks.

$$\text{Accuracy per check} = \frac{1+4}{1+5} = \frac{5}{6} \approx 83.3\%.$$

To count every conversation equally, give it one success only if all its checks are correct. The short conversation succeeds, and the longer one fails because of its single wrong check.

$$\text{Fraction of fully correct conversations} = \frac{1}{2} = 50\%.$$

The conversation-level fraction answers whether the entire request was handled correctly. The check-level accuracy gives the longer conversation five times as much weight as the short one because it contains five checks.

2. Compare two kinds of error

Treat an update as positive. A true positive is an update the model finds, and a false negative is an update it misses. A false positive is a lookup the model flags as an update, and a true negative is a lookup it correctly identifies. Subtract each model's true positives from the six actual updates to get its missed updates. Subtract its false positives from the fourteen actual lookups to get its correctly identified lookups. The confusion matrices are shown side by side.

Actual class	A predicts update	A predicts lookup	B predicts update	B predicts lookup
Update	3	3	5	1
Lookup	1	13	4	10

Accuracy divides the number of correct predictions by all 20 examples. Precision divides the number of updates found by the number of predicted updates. Recall divides the number of updates found by the six actual updates.

Metric	Model A	Model B
Accuracy	$(3+13)/20 = 80\%$	$(5+10)/20 = 75\%$
Precision	$3/(3+1) = 75\%$	$5/(5+4) \approx 55.6\%$
Recall	$3/(3+3) = 50\%$	$5/(5+1) \approx 83.3\%$

Model B misses two fewer updates and raises three more false alarms. Multiply each error count by its cost to obtain the total cost for each model.

$$C_A = 3c_{miss} + c_{false}, \qquad C_B = c_{miss} + 4c_{false}.$$

Model B costs less when the savings from two fewer misses exceed the cost of three extra false alarms. Because $c_{false} > 0$, we can divide both sides of the resulting inequality by $2c_{false}$.

$$C_B < C_A \Leftrightarrow 3c_{false} < 2c_{miss} \Leftrightarrow \frac{c_{miss}}{c_{false}} > \frac{3}{2}.$$

The costs are equal at a ratio of 3/2. Model A costs less below that ratio.

3. Make two reliability points

Keep each score with its label as you assign it to a bin. The horizontal coordinate is the mean predicted probability, and the vertical coordinate is the observed fraction of positives. Since the labels are zero or one, their mean gives that fraction.

Scores in bin	Labels in the same order	Mean score x	Observed rate y
$0.1, 0.2, 0.4$	$0, 1, 0$	$(0.1 + 0.2 + 0.4)/3 = 7/30$	$(0 + 1 + 0)/3 = 1/3$
$0.6, 0.8, 0.9$	$1, 1, 0$	$(0.6 + 0.8 + 0.9)/3 = 23/30$	$(1 + 1 + 0)/3 = 2/3$

The lower bin gives the point $(7/30, 1/3) \approx (0.233, 0.333)$. It lies above the diagonal $y = x$ because the observed rate exceeds the mean score. The upper bin gives $(23/30, 2/3) \approx (0.767, 0.667)$. It lies below the diagonal and is the bin whose mean score exceeds its observed rate.

4. Resample paired outcomes

Each difference is the correctness of B minus the correctness of A on the same example. It is $+1$ if only B is correct, -1 if only A is correct, and 0 if the models have the same outcome. Averaging these differences gives the accuracy difference, B minus A.

$$\text{Original mean} = \frac{0 + 1 + 1 - 1 + 0}{5} = \frac{1}{5} = 0.20.$$

The sampled indices $(2, 2, 4, 5, 1)$ select the second example twice, followed by the fourth, fifth, and first examples. The selected differences are $(1, 1, -1, 0, 0)$.

$$\text{Sampled mean} = \frac{1 + 1 - 1 + 0 + 0}{5} = \frac{1}{5} = 0.20.$$

Both means show a 20-percentage-point advantage for B. They happen to agree for this draw. Using the same index list keeps each model's result paired with the other model's result on the same example, including duplicates. Separate lists would lose that pairing. One model could draw more easy examples than the other, so a resampled difference could reflect the different mixes of examples as well as the models' performance.

9. Separate length from evidence placement (optional)

Ask the model to return the invoice number named in a report. A short fixture might contain the sentence *The total for order P27 was settled under invoice IN-843*. Other passages can contain plausible invoice numbers for other orders, but only that sentence answers the question about P27. Put the question at the end and score the invoice number exactly after applying a fixed normalization such as trimming surrounding whitespace.

Build a grid with report bodies of 512, 2,048, and 8,192 tokens before the question. In each row, place the beginning of the evidence at approximately 10%, 50%, and 90% of the body. Measure positions after applying the model's tokenizer. Record the exact token indices and the evidence-to-question distance. A position fraction is a rule for constructing examples. The measured distance tells us how far back the model must read.

Report body length	Early	Middle	Late
512 tokens	10%	50%	90%
2,048 tokens	10%	50%	90%
8,192 tokens	10%	50%	90%

Within a row, create three versions of each report. Keep the total token count, question, correct answer, evidence wording, and number and wording of distractors fixed. Move the evidence by swapping it with neutral text of the same token length at the intended position. Keep the distractors in the same locations and verify that the swap did not alter the tokenized length or create a second answer. Use many underlying reports and vary the correct invoice number across them. Compare exact-match accuracy across positions on these matched reports. This comparison measures the effect of moving the evidence within a report of the same length.

For a second experiment, hold the distance from the **end** of the evidence span to the beginning of the question at 128 tokens. Build one suffix consisting of the evidence, the 128 intervening tokens, and the question. Use the same suffix for all three report lengths. Tokenize each complete formatted prompt and verify that exactly 128 tokens lie between the evidence span and the question. Insert additional neutral material **before** the evidence as the report grows. Hold the distractor inventory fixed or explicitly define the filler to contain no candidate invoice numbers. In this experiment, the evidence sits later in relative terms in longer reports, but the question is always the same absolute distance away. The two experiments answer different questions. The first tests position at a fixed length. The second tests additional context at a fixed retrieval distance.

Report the number of test reports and paired accuracy differences for both comparisons. Keep variants of each underlying report together when splitting the data or resampling uncertainty. If the longest examples exceed a system's supported context, report that constraint and the number affected. The test needs the same complete input to compare evidence retrieval.

10. Test one change at a time (optional)

Suppose system B responds faster than system A in a measured workload. If the two systems have different mixers, training data, and numerical precision, the supported claim is that **the measured B configuration had lower response latency on that workload and hardware**. The observed difference combines the effects of these settings and their interactions. For example, lower precision can change which kernel runs and how many bytes it reads. The comparison does not isolate the mixer.

To study the mixer, build two variants of one model. Keep the tokenizer, input formatting, training and validation splits, number of training tokens, optimizer, evaluation procedure, and stopping rule the same. Use the same stack depth, residual width, feed-forward blocks, and head. Change only the designated mixer layers. Match the parameter budget where the designs permit and report remaining differences. Train each variant with the same set of several seeds and save checkpoints with the same validation rule. This measures the consequences of replacing the mixer under the specified training recipe.

Evaluate both variants on the same labeled examples. Report task accuracy and uncertainty over matched examples, and report variation across training seeds separately. For a task with a defined answer length, use a fixed answer format and decoding rule. Include results at several prompt lengths if mixer behavior is expected to change with the length of the history.

For latency, run both implementations on the same device and software stack with the same input, batch size, precision, and decoding workload. Warm up the kernels, synchronize before and after a timed region, alternate the order, and report the distribution across repeated trials. Record prefill time and time per decoding step separately. For the performance-only run, feed a fixed sequence of continuation tokens to both models so a model that stops early does not appear faster simply because it did less work. On the target workload, also report user-facing time to first token, from submission to the first returned token, and end-to-end latency, from submission to completion. Raw prefill time remains a separate diagnostic.

Measure persistent history bytes at the same prefix lengths. Count the live key and value arrays and any recurrent or convolutional state that remains between steps. Separately report allocated capacity when it differs from live payload, model weight bytes, and peak temporary workspace. This distinction explains a case in which one mixer uses a smaller history but still needs more scratch memory during prefill. An ablation can support a claim about the mixer swap under these controls. A comparison between release packages remains a separate systems result.

11. Design a reproducibility check (optional)

Two aggregate accuracies are a starting point. To recompute the result, ask for the dataset version, labeled evaluation rows, and the split manifest. The manifest should identify the unit scored, the stable example ID, its group ID if examples share a customer or document, and the assigned split. Ask how groups and repeated templates were assigned, which records were excluded, and which data were used to choose hyperparameters or a checkpoint. Check that no group appears in more than one split and that the reported evaluation rows match the frozen test manifest.

Ask for one prediction record per model per example, with the example ID, gold label or a link to the gold row, final predicted label, and any scores needed to replay thresholding. Request the threshold, label normalization, missing-prediction rule, and metric definition. Verify that the IDs are unique and that both sets match the manifest exactly. Then join by ID and recompute each accuracy and the difference. A positional join can silently compare one model's prediction with the label of a different example, even when both files contain the right number of rows.

For a paired interval, make a vector of B's correctness minus A's correctness on each aligned example. Its average is the observed accuracy difference. In a bootstrap replicate, draw the same examples with replacement for both systems. If rows are repeated checks from a customer or another dependent source, draw whole groups instead and keep the statistic consistent with the claim. For example, a per-check difference divides by the number of sampled checks, including all checks in repeated groups. Record the sampling unit, number of replicates, seed, and interval rule. This interval describes variation from resampling the evaluation cases for the two fitted systems.

Assessing variation across training runs requires independently trained checkpoints and their per-example predictions. Ask for training seeds, data-order and initialization controls, model revisions, training recipes, and how checkpoints were selected. Train both methods under matched seed and data conditions when feasible, then compute the paired evaluation difference separately for each pair of runs. The spread across those differences measures sensitivity to training, while a bootstrap on predictions from one pair measures sensitivity to the test sample. Neither number supplies the other.